

● Engenharia de Software

Prof. Esp. Otávio Cardoso E. S. Neto, PMP, MBA

○ Teoria geral de sistemas.

○ Conceitos de análise de sistemas.

○ Análise estruturada de sistemas.

○ Análise essencial de sistemas.

○ Ciclo de vida de um projeto.

○ Ferramenta de análise de sistemas.

○ Critérios em projetos de sistemas.

○ Estágios e objetivos do projeto.

○ Técnicas de documentos.

Mitos de Software;

Conceitos dos Métodos de Desenvolvimento de Software(Clássico, Iterativo
Incremental, Espiral, etc);

Teoria Geral de Sistemas

Sistema

O sistema é um conjunto de partes interagentes e interdependentes que, conjuntamente, formam um todo unitário com determinado objetivo e efetuam determinada função (OLIVEIRA, 2002, p. 35).

Sistema pode ser definido como um conjunto de elementos interdependentes que interagem com objetivos comuns formando um todo, e onde cada um dos elementos componentes comporta-se, por sua vez, como um sistema cujo resultado é maior do que o resultado que as unidades poderiam ter se funcionassem independentemente (ALVAREZ, 1990, p. 16).

Ex.: Sistema Operacional, Sistema Nervoso, Sistema Sanguíneo



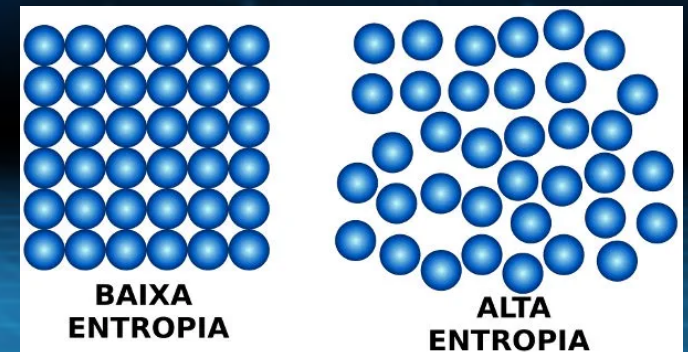
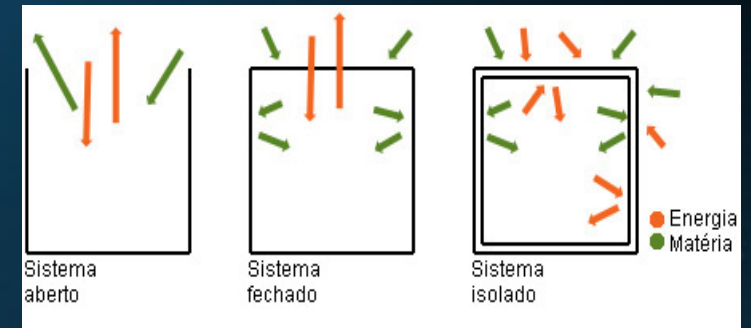
Teoria Geral de Sistemas

Ambiente

O ambiente de um sistema é um conjunto de elementos que não fazem parte do sistema, mas que podem produzir mudanças no estado do sistema.

Entropia

- 2ª Lei da termodinâmica
- Em um sistema isolado, a entropia (medida da desordem ou da dispersão de energia) tende a aumentar ao longo do tempo.
- Em sistemas abertos (que trocam energia ou matéria com o meio), é possível haver redução local de entropia — muitas vezes chamada, em biologia e teoria da informação, de negentropia — mediante entrada de energia ou informação.
- Na teoria da informação, entropia é uma medida da incerteza ou imprevisibilidade de um conjunto de dados. Quanto maior a entropia, menor a ordem e maior a quantidade de informação necessária para descrever o sistema.



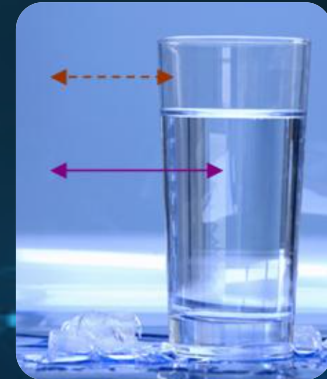
Teoria Geral de Sistemas

Sistemas Abertos

A teoria de sistemas afirma que estes são abertos e sofrem interações com o ambiente onde estão inseridos.

Ex.: Uma empresa é caracterizada como um sistema aberto, pois sofre interações e flutuações de seu ambiente interno (departamentos, processos, recursos humanos etc.) e do ambiente externo (economia, política, meio ambiente etc.).

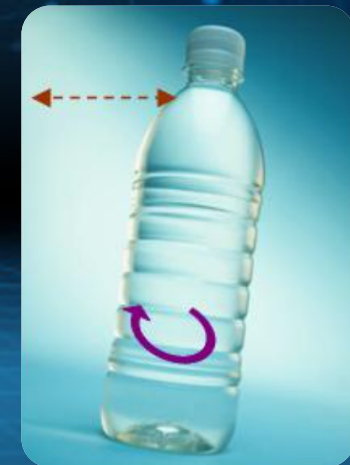
Sistemas biológicos, sociais: plantas, homens, organizações, sociedade.



Sistemas fechados

Esses sistemas são aqueles que não sofrem influência nem influenciam o meio ambiente no qual estão inseridos, de tal forma que ele se alimenta dele mesmo.

Ex.: Máquinas em geral, relógio, termômetro



Teoria Geral de Sistemas

Sistemas Isolados

Não ocorre troca de matéria (massa) nem de energia com o exterior (ambiente envolvente)

Não interagem com ambiente em que estão inseridos



Exemplos no mundo da tecnologia

Sistema abertos

Servidores Web (e.g., Apache, Nginx) – Recebem requisições HTTP e enviam respostas, interagindo constantemente com usuários e outros sistemas.

Redes Peer-to-Peer (P2P) – Como o BitTorrent, onde nós trocam dados entre si sem um controle centralizado.

APIs Públicas (e.g., Twitter API, Google Maps API) – Permitem a integração com outros sistemas e serviços externos.

Sistema Fechados

Sistemas Embarcados em Dispositivos Médicos (e.g., marcapasso inteligente) – Processam dados e respondem a estímulos, mas não permitem modificação física externa.

Controladores Lógicos Programáveis (CLPs) em Indústrias – Operam em ambientes controlados, recebendo entradas de sensores e enviando comandos a atuadores, mas sem interação direta com usuários.

Sistema isolados

Computadores Desligados e Lacrados – Sem conexão com redes ou fontes de energia.

Sistemas em Ambientes de Teste "Air-Gapped" – Máquinas isoladas fisicamente para análise de malware (sem internet ou redes externas).

Dispositivos de Armazenamento Offline (e.g., HDs em cofres à prova de fogo) – Dados armazenados sem possibilidade de acesso remoto ou modificação.

Teoria Geral de Sistemas

Componentes de Um Sistema



Teoria Geral de Sistemas

Componentes de Um Sistema

Entrada: envolve a captação e reunião de elementos que ingressam no sistema para serem processados

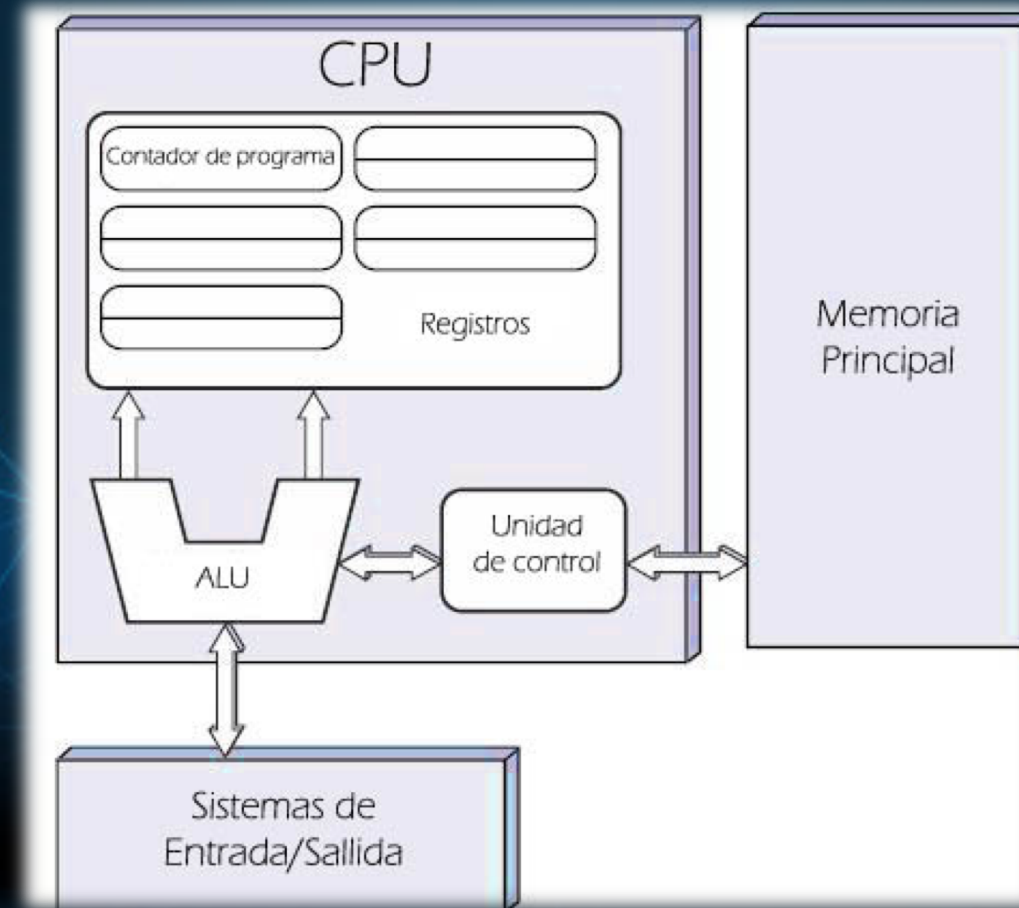
Processamento: envolve processos de transformação que convertem insumos (entradas) em produto.

Saída: envolve a transferência de elementos produzidos por um processo de transformação até seu destino final

Retroalimentação (ou feedback): reintrodução de uma saída sob a forma de informação.

Teoria Geral de Sistemas

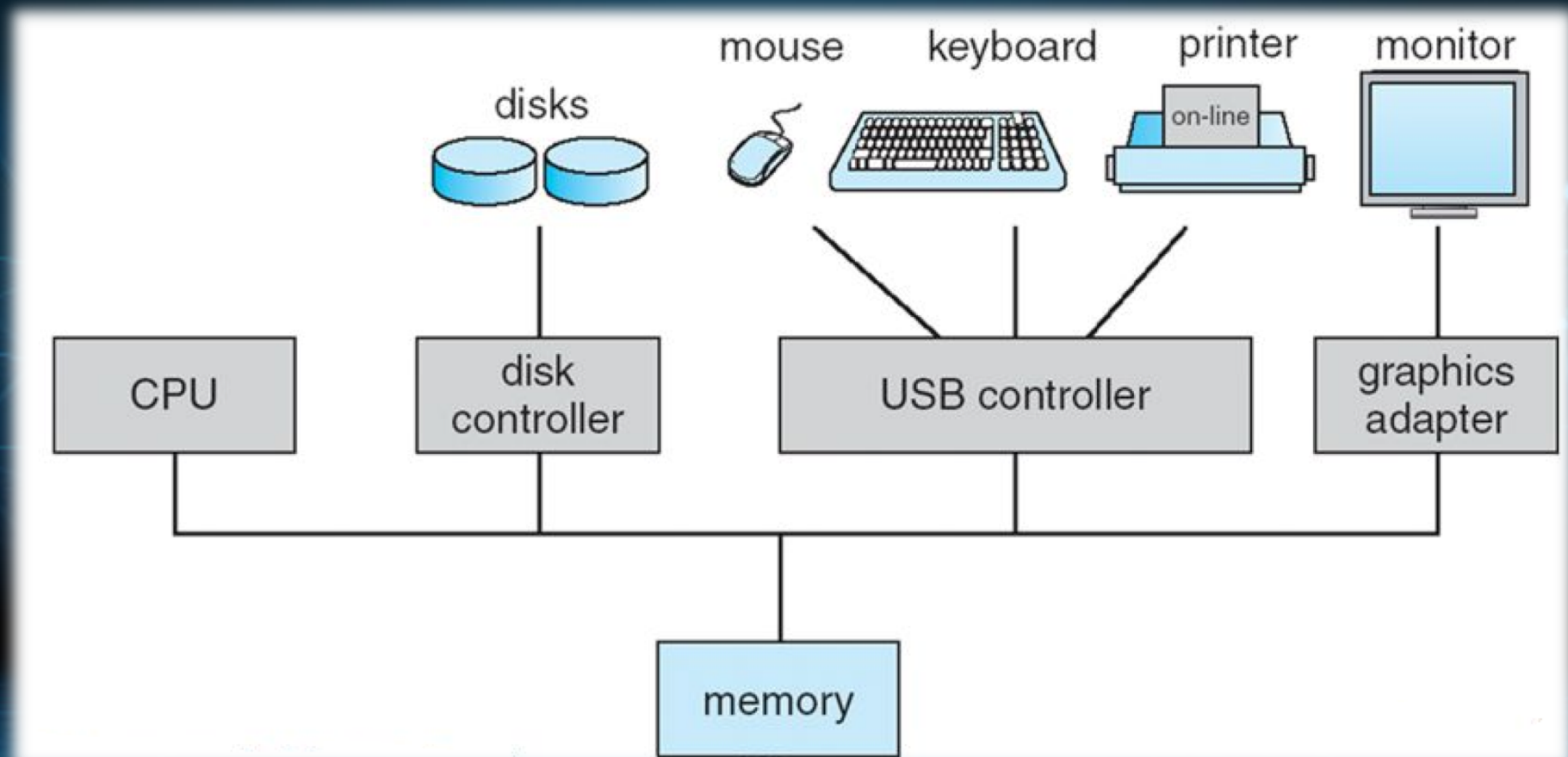
Elementos de Um Sistema Computacional



Arquitetura de Von Neumann

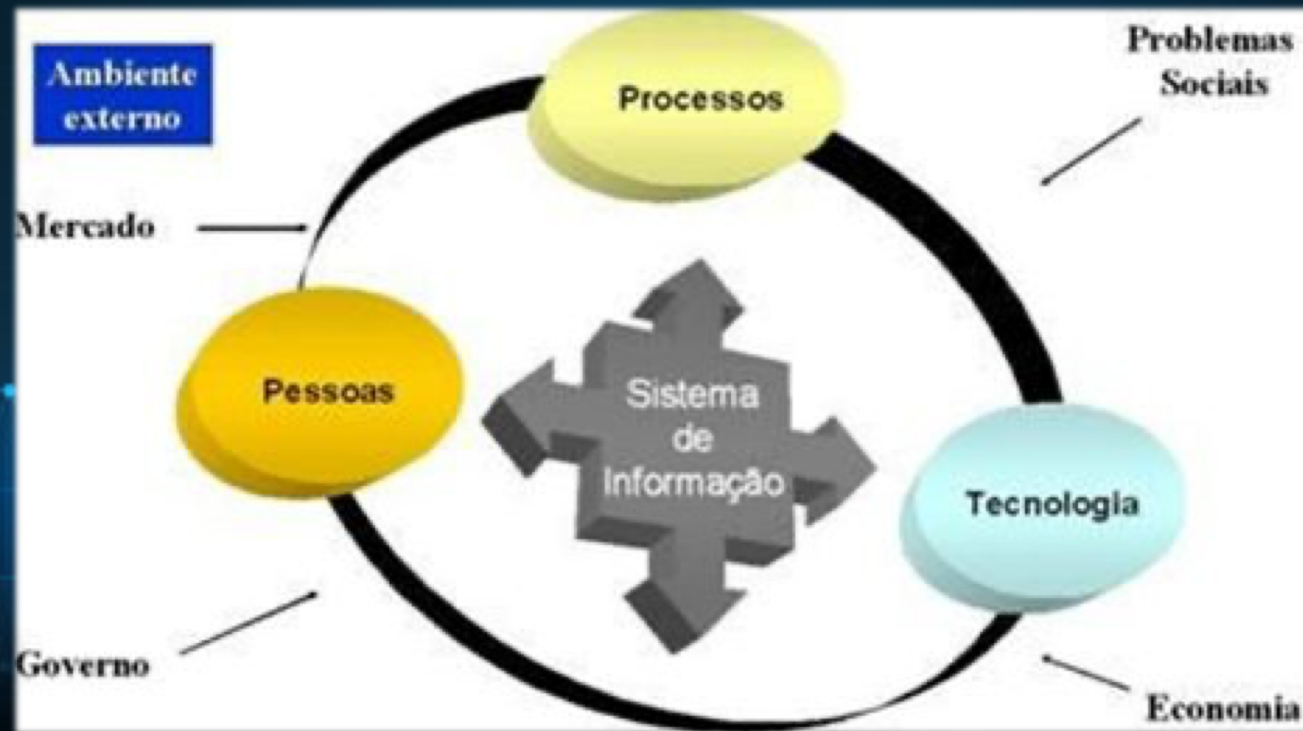
Teoria Geral de Sistemas

Elementos de Um Sistema Computacional



Teoria Geral de Sistemas

Sistema da Informação



Exemplo de um sistema de informação e seus componentes

Fonte: <https://queconceito.com.br/sistema>

Teoria Geral de Sistemas

Sistema da Informação

Informações são dados oferecidos de uma forma significativa e útil para os indivíduos.

A informação pode ser transmitida de pessoa para pessoa, seja de maneira escrita (como livros, cartas, entre outros), como também armazenada eletronicamente (em arquivos de computador ou dispositivos móveis), podendo, nesse caso, ser transmitida infinitamente. A informação ainda pode ser transmitida por meio de conversas ou através de filmes, séries, documentários, etc.

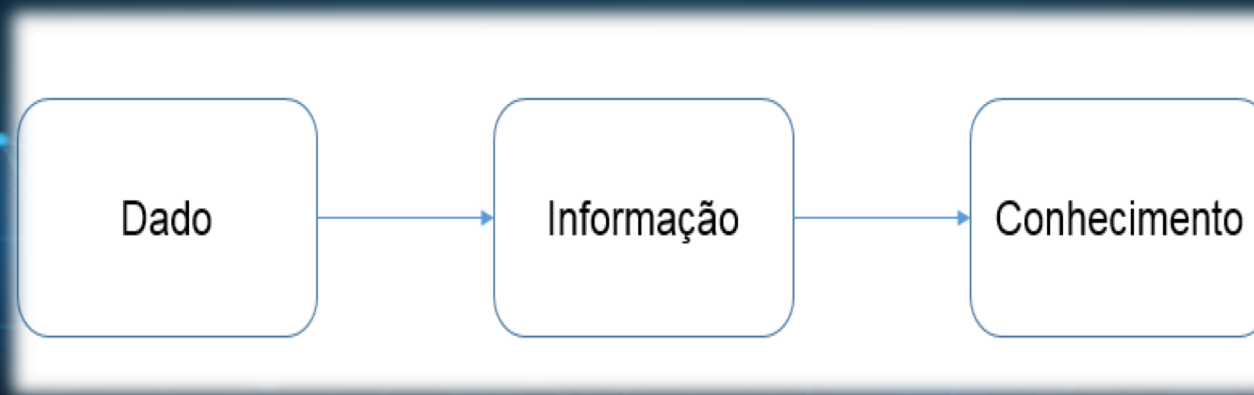
Sistemas de Informação abrange pessoas, máquinas e/ou métodos organizados para coletar, armazenar, processar, transmitir e disseminar dados que representam informação para o usuário e/ou cliente (RAINER, L. Kelly; CASEY, G. Cegielski. Introdução a Sistemas de Informação. São Paulo: LTC, 2015.).

Um sistema de informação pode ser definido como um conjunto de componentes inter-relacionados trabalhando juntos para coletar, recuperar, processar, armazenar e distribuir informações, com a finalidade de facilitar o planejamento, o controle, a coordenação, a análise e o processo decisório em organizações (Laudon; Kenneth C.; Laudon; Jane Price (1999). Sistema da Informação com Internet. [S.l.: s.n.] 4 páginas).

Teoria Geral de Sistemas

Sistema da Informação

Concluimos que, a informação é o resultado da análise e organização dos dados com o intuito de conferir significado dentro de um contexto.



Ciclo de vida de um Projeto

O ciclo de vida é a estrutura contendo processos, atividades e tarefas envolvidas no desenvolvimento, operação e manutenção de um produto de software.

Para a **NBR ISO/IEC 12207:1998**: “Estrutura contendo processos, atividades e tarefas envolvidas no desenvolvimento, operação e manutenção de um produto de software, abrangendo a vida do sistema, desde a definição de seus requisitos até o término de seu uso.”

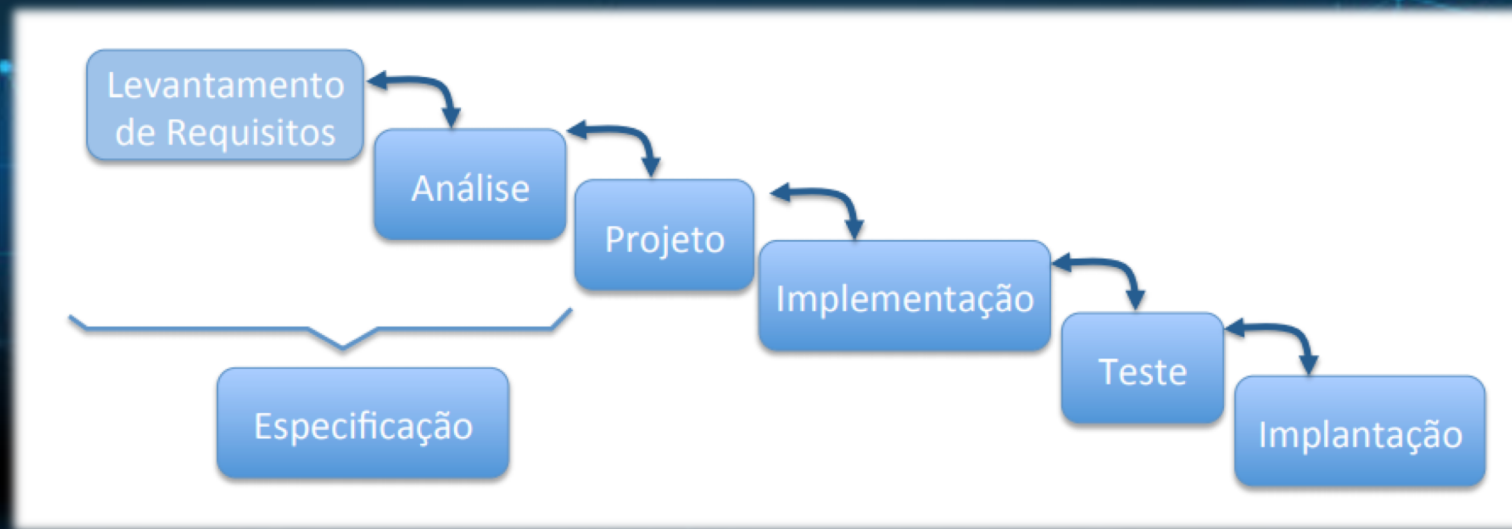
Processo de software é o conjunto de atividades que constituem o desenvolvimento de um sistema computacional. Estas atividades são agrupadas em fases, como: definição de requisitos, análise, projeto, desenvolvimento, teste e implantação.



Ciclo de vida de um Projeto

Um processo contém diversas atividades divididas em fases.

O encadeamento específico dessas fases dá-se o nome de Modelo **Prescritivos** ou **Não Prescritivos** de Processo



Ciclo de vida de um Projeto

Prescritivos

Seguem uma forma mais definida e sistemática de se construir software.

Surgiram para estruturar atividades de um processo

Prescrevem um conjunto de elementos e como esses elementos se inter-relacionam

Definem quais produtos (artefatos) tem que ser gerados

Tipos de Modelos Prescritivos:

- Cascata
- Modelo Incremental
- Modelo evolucionário - espiral, prototipação

Não Prescritivos

Foco maior nos resultados do que no processo.

Modelos ágeis

- Scrum
- Extreme Programming (XP)

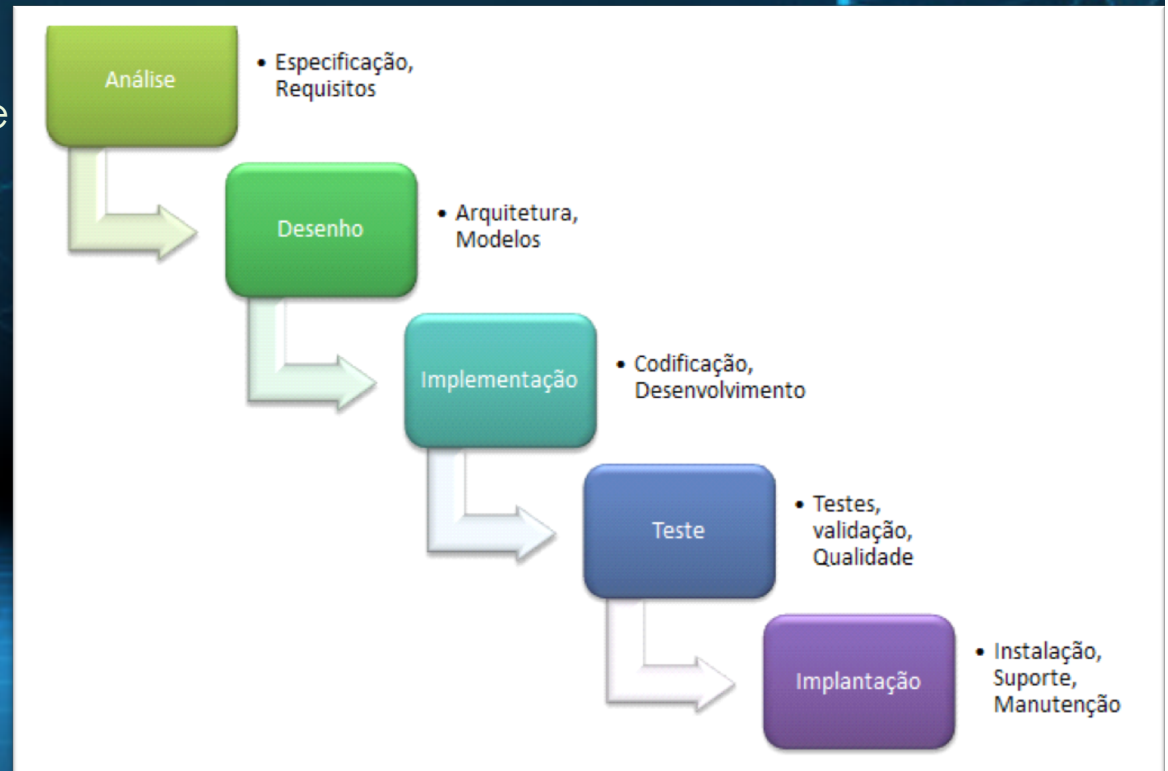
Ciclo de vida de um Projeto

Modelo em Cascata/waterfall

Divide todo o processo de desenvolvimento de software em fases separadas e sequenciais onde o desenvolvimento movimenta-se somente num sentido, de modo que as etapas não podem ser repetidas.

Somente quando uma fase estiver concluída a outra fase poderá iniciar.

Neste modelo nenhum componente do sistema será entregue até a proximidade final do projeto.



Ciclo de vida de um Projeto

Modelo Espiral

É uma combinação dos modelos iterativo e sequencial com ênfase na análise de riscos e no planejamento.

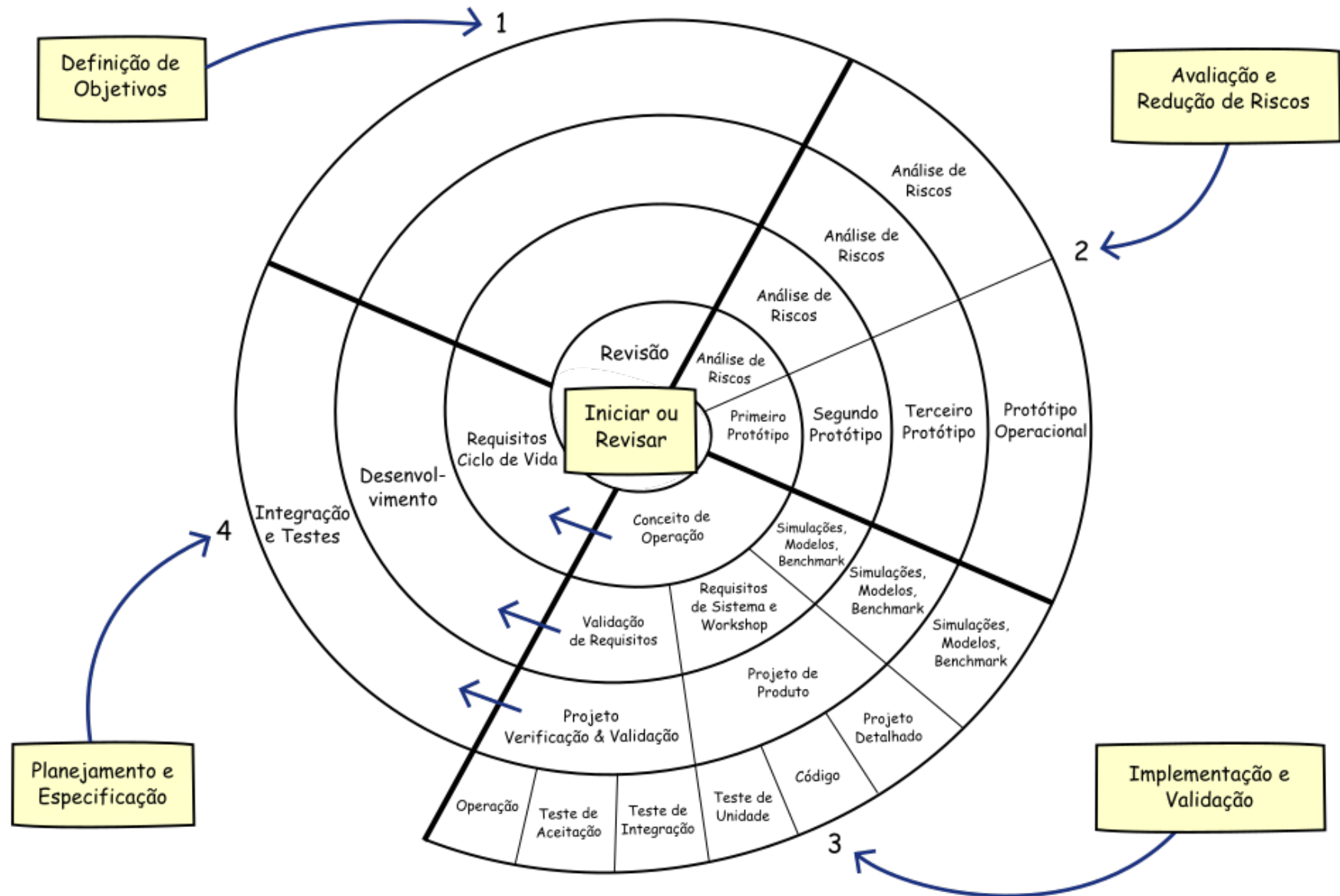
O sistema de software é entregue em versões, onde cada versão passa por cada etapa do ciclo de desenvolvimento. Dessa forma o produto final é entregue rapidamente.

Determina um ciclo de atividades dividido em 4 estágios:

1. Determina objetivos, soluções alternativas e restrições.
2. Analisa os riscos do estágio anterior
3. Atividades da fase de desenvolvimento : design, especificação, codificação e verificação
4. Revisão das etapas anteriores e o planejamento da próxima fase

Ciclo de vida de um Projeto

Modelo Espiral



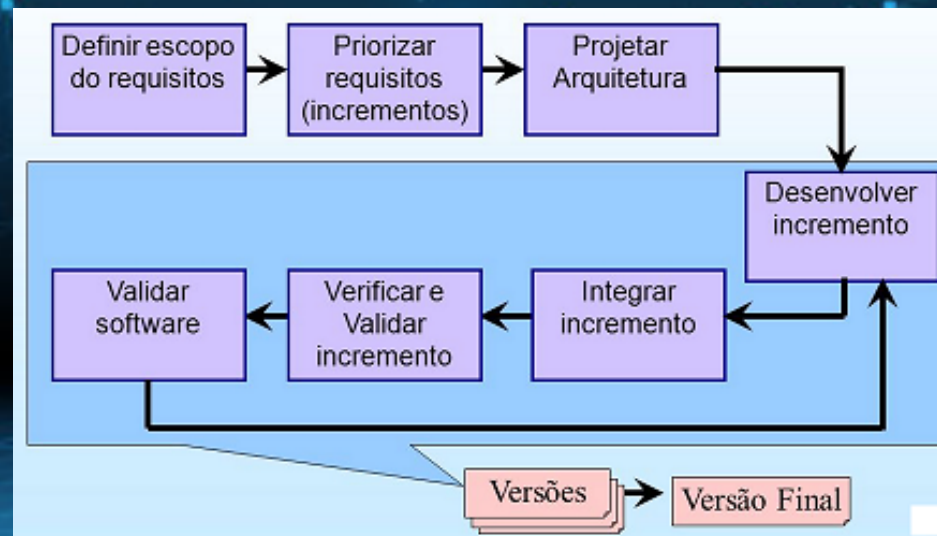
Ciclo de vida de um Projeto

Modelo Incremental

Desenvolve várias partes do sistema em paralelo, e as integra quando estiverem completas.

O desenvolvimento é dividido em 'incrementos' que produzem o sistema de forma incremental até a sua versão final.

Cada incremento passa por todo o ciclo de desenvolvimento do software produzindo um sistema funcional embora não atenda todos os requisitos.



Ciclo de vida de um Projeto

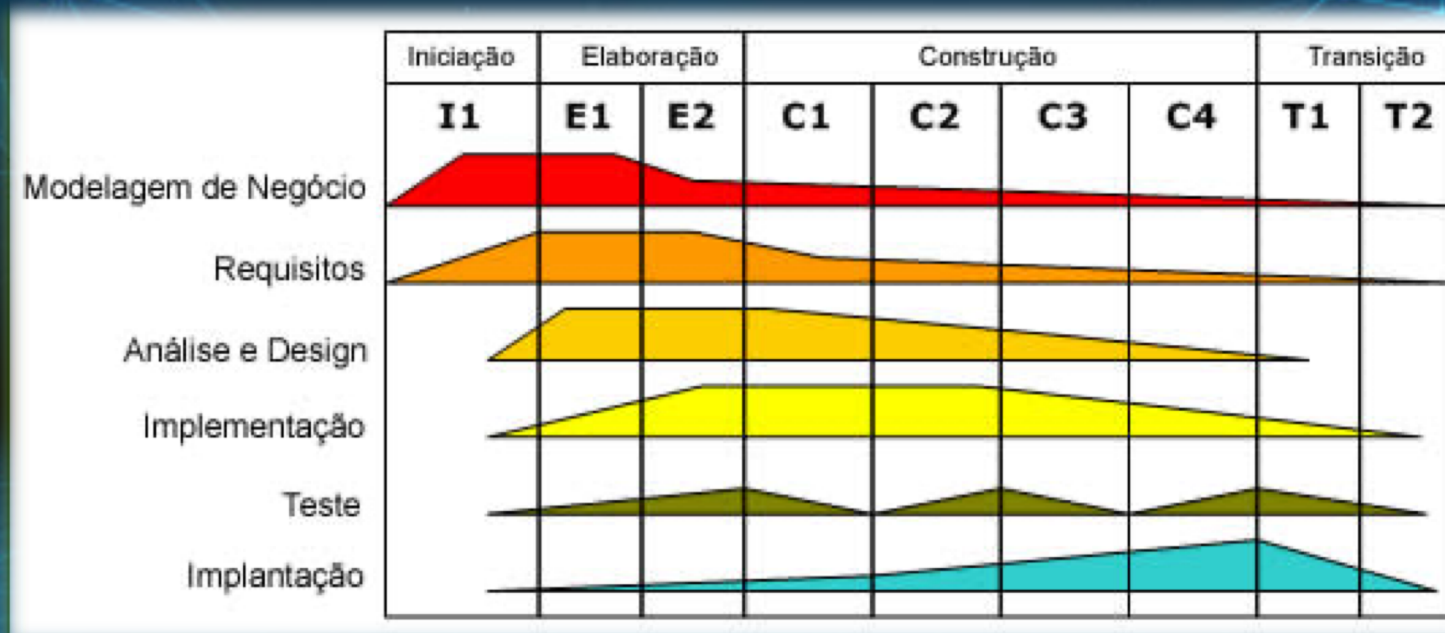
Prototipagem

Constrói um protótipo do produto de software para mostrar suas funcionalidades de forma limitada.

É usado para permitir que os usuários avaliem propostas de desenvolvedores testando-as antes da implementação final do produto.

Ajuda a compreender as exigências específicas do usuário que não podem ter sido consideradas pelos desenvolvedores durante a fase de concepção do produto.





Metodologias Ágeis

Cada iteração tem como objetivo entregar uma parte do projeto, que normalmente dura de 1 a 4 semanas, e inclui todas as fases para implementá-lo como levantamento de recursos e requisitos, projeto, desenvolvimento de código, testes e documentação.

Ao final de cada iteração deve haver uma entrega ao cliente, que inclua um conjunto de novas funcionalidades, uma nova versão de software.

Após essa entrega há um novo processo de comunicação com o cliente e então são definidas quais deverão ser as novas entregas.



Ciclo de vida de um Projeto

Metodologias Ágeis



Análise e Projeto de Software

Projeto de Software

Projetar software é uma atividade complexa.

Grande parte desta complexidade é inerente à própria natureza do software.

Computadores são algumas das máquinas mais complexas já construídas pelo homem. Eles possuem um número muito grande de estados. Sistemas de software possuem algumas ordens de grandeza a mais de estados que computadores.

Além de serem complexos por natureza, sistemas de software são constantemente sujeitos a mudança.

Outra razão para essa “pressão” por mudanças se deve ao fato do software ser uma entidade flexível e que pode ser mudado com muito mais facilidade do que produtos manufaturados.

Análise e Projeto de Software

Projeto de Software

“Projetar sistemas de software significa determinar como os requisitos funcionais são implementados na forma de estruturas de software” [PETERS; PEDRYCZ, 2001].

“...projeto é a única maneira com a qual se consegue traduzir com precisão os requisitos do cliente em um produto de software ou sistema finalizado”. [PRESSMAN, 2005].



Análise e Projeto de Software

Projeto de Software

O processo de desenvolvimento de software está inserido no ciclo de vida do software.

Para a criação de modelos de ciclo de vida de software estabelece que os principais processos da fase de desenvolvimento de software são: Requisitos, Projeto e Implementação [IEEE, 1997].

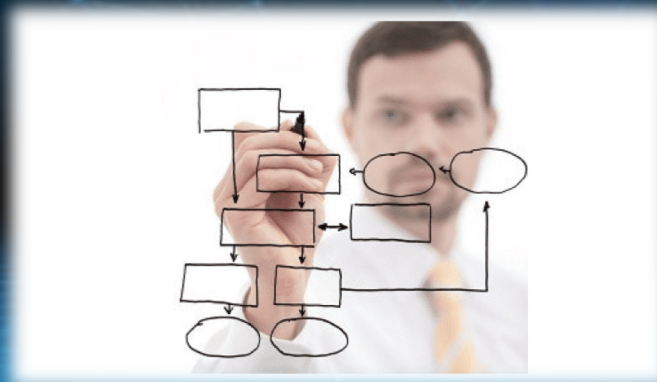
Nos modelos de ciclo de vida, a atividade de projeto recebe o resultado da atividade de requisitos e gera um modelo para a atividade de implementação ou codificação.

Análise e Projeto de Software

Projeto de Software – Arquitetura de software

Durante o projeto de software é criada a estrutura interna de um sistema de software. Esta estrutura interna é chamada de arquitetura.

Arquitetura de software pode ser definida como a estrutura ou organização dos componentes do programa (módulos), a maneira com a qual estes componentes interagem e a estrutura da informação que é usada por estes componentes [PRESSMAN, 2005].



Análise e Projeto de Software

Projeto de Software

O grande objetivo de um projeto de software é levar a um produto de software que tenha qualidade.

Para conseguir alcançar os objetivos de qualidade do produto de software gerado, um projeto de software deve ter qualidade.

Para Pressman (2005), existem diretrizes para a qualidade de um projeto de software, e entre elas estão:

- Um projeto deve ser modular;
- Um projeto deve conter representações distintas para dados, arquitetura, interfaces e componentes;
- Um projeto deve levar a componentes que possuam características de independência funcional;
- Um projeto deve levar a interfaces que reduzam a complexidade das conexões entre os componentes e o ambiente externo.

Análise e Projeto de Software

Projeto de Software

Pressman (2005) enuncia princípios de projeto de software, para atingir certas diretrizes de qualidade de software:

- 1.O projeto deve ser rastreável para o modelo de análise;
- 2.Sempre considere a arquitetura do sistema a ser construído;
- 3.O projeto dos dados é tão importante quanto o projeto das funções de processamento;
- 4.As interfaces (ambas internas e externas) devem ser projetadas com cuidado;
- 5.A interface com o usuário deve estar sintonizada com as necessidades do usuário final;
- 6.O projeto, no nível dos componentes, deve ser funcionalmente independente;
- 7.Componentes devem ser fracamente acoplados entre eles e com o ambiente externo;
- 8.As representações (modelos) de projeto devem ser entendidas facilmente;
- 9.O projeto deve ser desenvolvido iterativamente.



Mitos do desenvolvimento de software

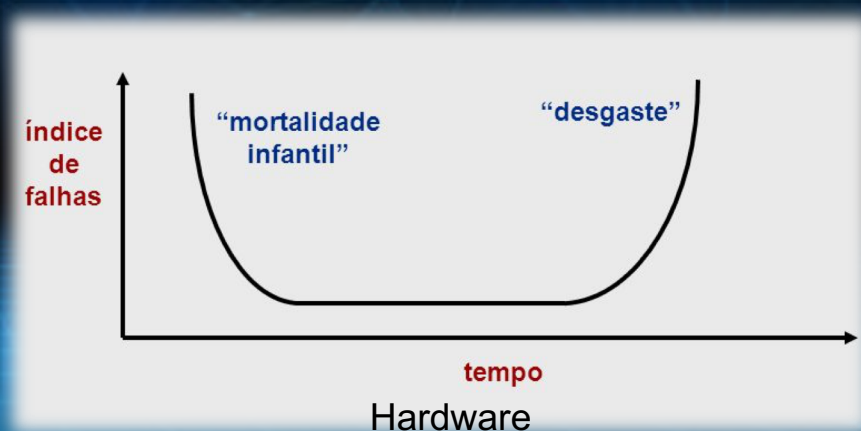
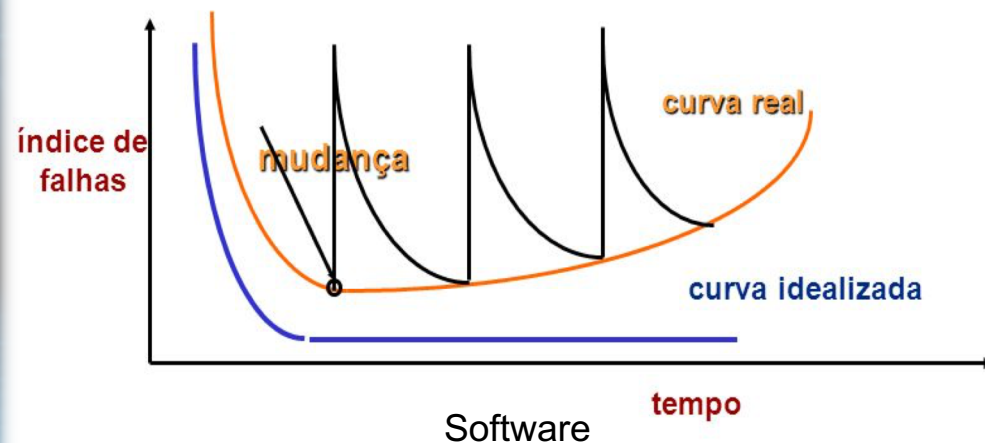
Os mitos de software são “falsas verdades” que existem no mundo da indústria de software

Características do Software

Software não é sensível aos problemas ambientes que fazem como que o hardware se desgaste.

Toda falha indica erro de projeto ou implementação.

Manutenção do SW é mais complicada que a do HW



Mitos do desenvolvimento de software

Aplicações de Software

Software de aplicações básicas

Software de aplicações de escritório

Software de Tempo Real

Software Comercial

Software Científico ou de Engenharia

Software Embutido

Software de Dispositivos Móveis

Software de Inteligência Artificial



Mitos do desenvolvimento de software

1º Mito – Temos padrões e regras

Já possuímos um manual com padrões e procedimentos para a construção de um Software.

Isso não é suficiente?

Realidade:

Será que o manual é usado?

É conhecido pela equipe?

É completo?

Desenvolvimento não é uma receita de bolo! Os clientes são diferentes, os projetos são diferentes, os programadores são diferentes!

Mitos do desenvolvimento de software

2º Mito – Temos bons hardwares

Possuímos computadores de última geração!

Realidade:

Só computadores não são suficientes.

Ferramentas e processos de engenharia de software devem ser o mais importantes.

Mitos do desenvolvimento de software

3º Mito – Atraso no desenvolvimento do projeto

Vamos contratar mais programadores!

Realidade:

O desenvolvimento de software não é um processo manufaturado (fabricado), assim, mais programadores acabam por atrasar ainda mais o processo.

“Nove mulheres não conseguem fazer um filho em um mês”

Mitos do desenvolvimento de software

4º Mito – Os requisitos mudam, mas é fácil fazer as alterações.

Os requisitos do software modificam-se continuamente, mas, as mudanças podem facilmente serem acomodadas.

Realidade:

O impacto da mudança varia de acordo com o tempo em que ela é introduzida.

Um atenção na definição inicial possibilita uma boa acomodação de solicitações de mudança.

Mudanças durante o projeto do Software impacta fortemente os custos, já que recursos foram comprometidos.

Outros impactos nos prazos, qualidade e novas incertezas geradas.

Mitos do desenvolvimento de software

5º Mito – Mito do Profissional

O trabalho se resume a escrever e colocar um programa em funcionamento

Realidade:

Quanto mais cedo se inicia o código, mais tempo é despendido no seu término: de 50 a 70% do esforço é despendido depois da primeira entrega.

Mitos do desenvolvimento de software

6º Mito – Não há como verificar a qualidade enquanto não colocar para rodar o programa

Enquanto o programa não estiver funcionando, não há como avaliar sua qualidade

Realidade:

A revisão técnica formal inicia-se desde a concepção do projeto, aplicando mecanismos de garantia da qualidade em todas as fases do desenvolvimento

Mitos do desenvolvimento de software

7º Mito – Não precisamos de planejamento, já sabemos o que fazer

O processo de planejamento fará com que criemos documentação volumosa que atrasará a execução do projeto, atrasando o cronograma

Realidade:

Planejamento é fundamental!

Muitas pessoas ainda confundem planejamento com “papelada”, mas não é!

Mesmo trabalhando-se em um time Agile, planejar é fundamental! A documentação do projeto será trabalhada na melhor metodologia adotada

Mitos do desenvolvimento de software

8° Mito – Terceirizar um projeto

Terceirizar um projeto é garantia de tranquilidade e nenhum trabalho

Realidade:

Quando um projeto é muito trabalhoso, requer know-how maior do que a sua equipe possui ou o cronograma está apertado, muitos optam pela terceirização achando que esta é uma garantia de tranquilidade e nenhum trabalho.

A terceirização fez o serviço direito? Comentou o código?

Documentou o que foi feito? Sua equipe tem pessoal para trabalhar nesse código?



DÚVIDAS?

“Boas coisas acontecem quando
você se dedica ao estudo!”

otavio.cardoso.neto@gmail.com